

Challenge Problem 1

Noel Craig, Kam Kelly

September 2026

https://github.com/Robo-Kam/CP1_I0Br05.git

1 Introduction

Overall idea of the robot warehouse program:

1. generate Robots with all of their initial attributes based on the size of the grid
2. run path planning for each robot
3. store robots in list
4. set robots with positions to a dictionary for the respective timestep
5. move robots in grid based on path planning
6. check if robot positions are compatible
7. if not compatible, move a robot back a position from the timestep history
8. repeat steps 5-7

The robots for the program were generated using a function in the grid class and all of the initialization of the robot information was done in the robot class. The structure of the program takes an input from the terminal to determine the size of the square grid n , that is a number between 3 and 10. The number of robots generated also followed the requirement to always be $2 * n$. When the robots were generated, they were immediately initialized to a list that was an attribute of the grid class. This initialization included the A* path planning algorithm[1] that utilized a euclidean distance heuristic. This algorithm only works marginally better for smaller n -values than Dijkstra, but significantly better as n increases due to the heuristic requirement of the algorithm.

2 Class Descriptions

2.1 grid class

Includes methods for plotting with matplotlib: plot robot position, plot goal position, plot distance from current position to goal, plot shape of each robot type, plot the planned path of each robot.

The Grid class keeps track of all movement history(position/safety conditions,etc) while the overall Grid python file is responsible for initializing graphs, plotting data on graphs, and changing plots with keybinds.

- HISTORY: The way position data for each robot is graphed can be found in the history list. This list contains many dictionaries with information gathered is initialized from the snapshot function and the history list is iterated from the timestamp sub.

- NEXT POSITION: This sub function takes the robot path list generated from the A^* method to index the next position per iteration. This is performed by finding the robot path and increasing the index of the position. This was initially a section of the run function but ran into many challenges indexing the timestamps.
- Conditions Check: This sub function takes takes the positional data of all robots and compares each position to pass the safety check to keep moving forward.
- RUN: This sub function is what appends the data from new moves for each robot to history. The conditional check is used before appending the data.
- DRAW SHAPE: Function for identifying each robot type and assigning a shape based on the type to plot. This uses the patches library from matplotlib to create/plot the shapes.
- DRAW GRID: Function responsible for initializing the plots without any data.
- CREATE GRID: Function responsible for creating the robots, adding robots, and appending data to the history. This way, the robot initialization can be done in a single line.
- PLOT GRID: Main nested function for all plotting. Initially, there were two just two separate plots for the robots but the assigned colors per plot did not match.
- DRAW ROBOT GRID: Draw robot grid is responsible for the plots of A^* Data. This is done by indexing the robot number from the robot list and plotting its path data from the robot class.
- DRAW TIMESTAMP: Function responsible for graphing all robot positions based on snapshot index. This was the easiest part of the code to implement after the snapshot/timestamp functions were complete.
- UPDATE: Function responsible for using keybinds to change the timestamp index or robot number. This is performed by indexing the timestamp or robot number and using that index to plot the updated data. This method was easier than creating a GUI toggle button on the matplotlib graph.
- MAIN: Running code for the grid. Uses an input to assign a grid size and robot list to create the plot. There is then an if statement to check the validity of grid dimensions. The Try/Except approach was considered but too many arguments were required to implement.

```

Set Grid Size, n
5

Drone: Triangle
Humanoid: Square
Driver: Circle

```

Figure 1: Example of terminal plot, input, and shape identification

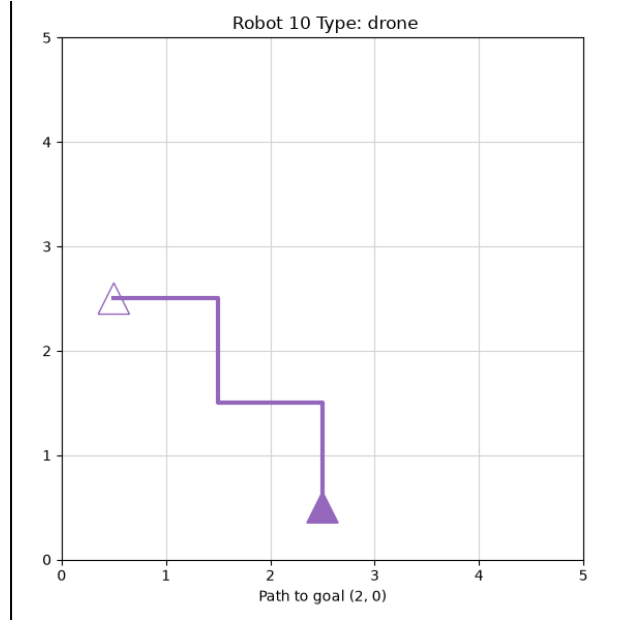


Figure 2: Example of planning plot displaying A^* method

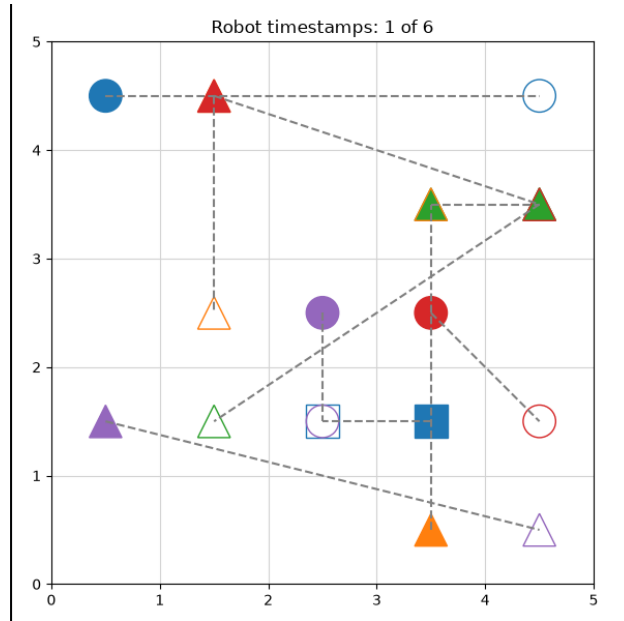


Figure 3: Example of Timestamp Plot with 5x5 Grid input

2.2 robot class

The attributes of the Robot class are as follows: (self, position, robotype, goal, distance, isFinished, griddim). More thorough descriptions of each of the attributes can be found as comments in the working code, but a brief explanation of the attribute along with its challenges are described here:

- **POSITION:** there was debate on whether there should have been a separate attribute for the start position aside from the position. We found that the start position attribute was not needed, because the position attribute was stored in the grid class for each separate timestamp. So, history[0] always

held the start position for each robot. The start position was generated using the randrange function from the random library to just be a value within the grid space for the x and y coordinates.

- **ROBOTYPE**: this attribute originally started as just a string to describe what robot type the specific robot that was generated. Having the robotype just be a string lead to some issues in implementing functions from the robot subclasses for drone, humanoid, and driver. So, we learned that the class could actually be stored to this attribute to more effectively call functions from the subclasses by using conditionals for the robotype attribute.
- **GOAL**: the way this attribute was generated was the same as the position attribute, however a condition was set to avoid the goal being in the same position as the initial robot position. This attribute was never changed unless the program was restarted.
- **DISTANCE**: this value was almost not used until the distance from the goal was needed for the conditional check function for the different robot classes. In that function it was used to determine which robot would proceed to the goal and which robot would remain in place.
- **ISFINISHED**: this value was helpful to optimize the code, and avoid rerunning functions when a robot was finished running. It was difficult to track this until the grid was being displayed.
- **GRIDDIM**: It was attempted in the beginning to not have a dependency from the grid class on the robot class, but there was too much of the initialization of the robot class that depended on the size of the grid. This was added after an unfortunate amount of troubleshooting and simplified the generation of all of the robots at the beginning of the program.

```
NAME: Robot_1
TYPE: <class 'Humanoid.humanoid'>
POSITION: (1, 1)
GOAL: (4, 2)
DISTANCE: 3.1622776601683795
ISFINISHED: False
GRID_DIM: 5
```

Figure 4: Example of robot generated after robot class initialization

Subclasses were made for the robot class for each different type of robot. These subclasses ended up having less information in them than expected. The major difference between the different subclasses was a function to check if the robot was in the same position as an incompatible robot. The only attribute that was different from the robot parent class was the 'isSafe' attribute. This attribute was just used as a returned boolean from the function in each of the subclasses checking whether a specific robot was in the same position as an incompatible robot.

References

- [1] Nikolaus Correll. *Robotic Mapping and Trajectory Generation*. Coursera. Accessed: August 2026. URL: <https://www.coursera.org/specializations/introduction-robotics-webots>.